



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Build a Real-Time Cyber Threat Detection Engine Using Hashing and Self-Balancing Trees

ASSIGNMENT REPORT

*Submitted in the partial fulfilment for the Course
of*

CSA0309 - Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

Submitted by

PRENA M [192524056]

SATHVIKA B [192525224]

Under the Supervision of

Dr. Uma Priyadarsini P.S

Saveetha School of Engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105 September

2026

A. Assignment Information

Particular	Details
Department	Department of Computer Science and Engineering
Programme	B.Tech – Artificial Intelligence and Machine Learning
Course Code & Course Name	CSA0309 – Data Structures
Academic Year / Batch	2026–2027
Faculty Name	Dr Uma Priyadarsini P.S
Assignment Title	Build a Real-Time Cyber Threat Detection Engine Using Hashing and Self-Balancing Trees
Date of Issue	31/08/2026
Date of Submission	02/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply the suitable Abstract Data Type for construction of the high-speed security log processing system. CO3: Make use of self-balancing trees (AVL/Red-Black Trees) for ordered, dynamically balanced storage of risk-ranked events. CO4: Make use of hashing, collision-resolution techniques, and priority queues for duplicate detection and high-risk event ranking in C. CO5: Develop a robust real-time cyber threat detection engine and evaluate its search performance under varying load factors and data distributions.
Bloom's Taxonomy Level	L4/L5 – Analyze / Evaluate / Create
SDG Mapping	SDG 9 – Industry, Innovation and Infrastructure; SDG 16 – Peace, Justice and Strong Institutions
Industry / Societal Relevance	Efficient processing of cybersecurity logs for rapid duplicate-IP detection, risk ranking and threat prioritization.

B. Assignment Problem / Challenge

A cybersecurity operations center receives security event logs such as login attempts, IP connections and access requests at high volume. Each event contains a source IP address, timestamp, event type and risk score. The challenge is to design an efficient C-based threat detection engine that can identify repeated IP addresses quickly, maintain events in risk order, surface high-risk events first, and remain efficient as the number of events increases.

The solution uses a hash table with chaining for average $O(1)$ duplicate-IP lookup, an AVL tree for dynamically ordered risk-based storage, and a max-heap priority queue for high-risk event ranking. Hash-table performance is also tested under different event loads.

C. Problem Statement

The proposed system processes a large stream of cybersecurity events with minimal latency. It must detect repeated IP addresses that may indicate brute-force attacks or scanning, rank suspicious events according to risk, and provide

ordered reports. The system should handle collisions and skewed data distributions without significant performance degradation.

The student develops a complete C program that integrates hashing, an AVL tree and a priority queue, followed by empirical performance testing of hash insertion and search operations.

D. Requirements and Constraints

Functional Requirements

- Read and store security event details.
- Detect duplicate or repeated IP addresses.
- Maintain events according to risk score.
- Display the highest-risk events first.
- Perform IP search efficiently.
- Generate performance measurements.

Performance Requirements

- Hash insertion/search: $O(1)$ average case.
- AVL insertion/search: $O(\log n)$.
- Priority queue insertion/deletion: $O(\log n)$.
- Support large event volumes with efficient memory use.

Technical Constraints

- Implementation language: C.
- Hash collision resolution: separate chaining.
- Self-balancing tree: AVL Tree.
- Priority queue: Max Heap.
- Performance measurement: `clock()` function.

Security, Reliability and Ethical Constraints

- Repeated IPs must be counted correctly.
- Collisions must not lose events.
- The system should avoid incorrect risk ordering.
- Security data should be handled only for legitimate monitoring purposes.

E. Student Work

1. Problem Understanding and Formulation

The problem is to build a high-speed security event processing engine using suitable data structures. Traditional linear search becomes inefficient as the number of events grows. Hashing provides fast IP lookup, AVL trees provide balanced ordered storage, and priority queues provide immediate access to the highest-risk event.

Expected Outcomes:

- Fast duplicate-IP detection.
- Risk-based event ordering.

- Immediate identification of the highest-risk event.
- Efficient processing for increasing event volumes.
- Performance comparison using measured execution time.

Available Data:

Field	Example
Source IP	192.168.1.10
Timestamp	10:00:01
Event Type	Login Failed
Risk Score	90

Assumptions:

- Risk scores range from 0 to 100.
- A higher score indicates greater risk.
- IP addresses are represented as strings.
- Hash collisions are handled using chaining.

2. Application of Course Knowledge

Data Structure / Technique	Application	Complexity
Hash Table	Duplicate IP detection and lookup	$O(1)$ average
Chaining	Collision resolution	Depends on chain length
AVL Tree	Ordered risk storage	$O(\log n)$
Max Heap	Highest-risk event retrieval	$O(\log n)$ deletion
Linked List	Collision chains	$O(n)$ worst case per bucket

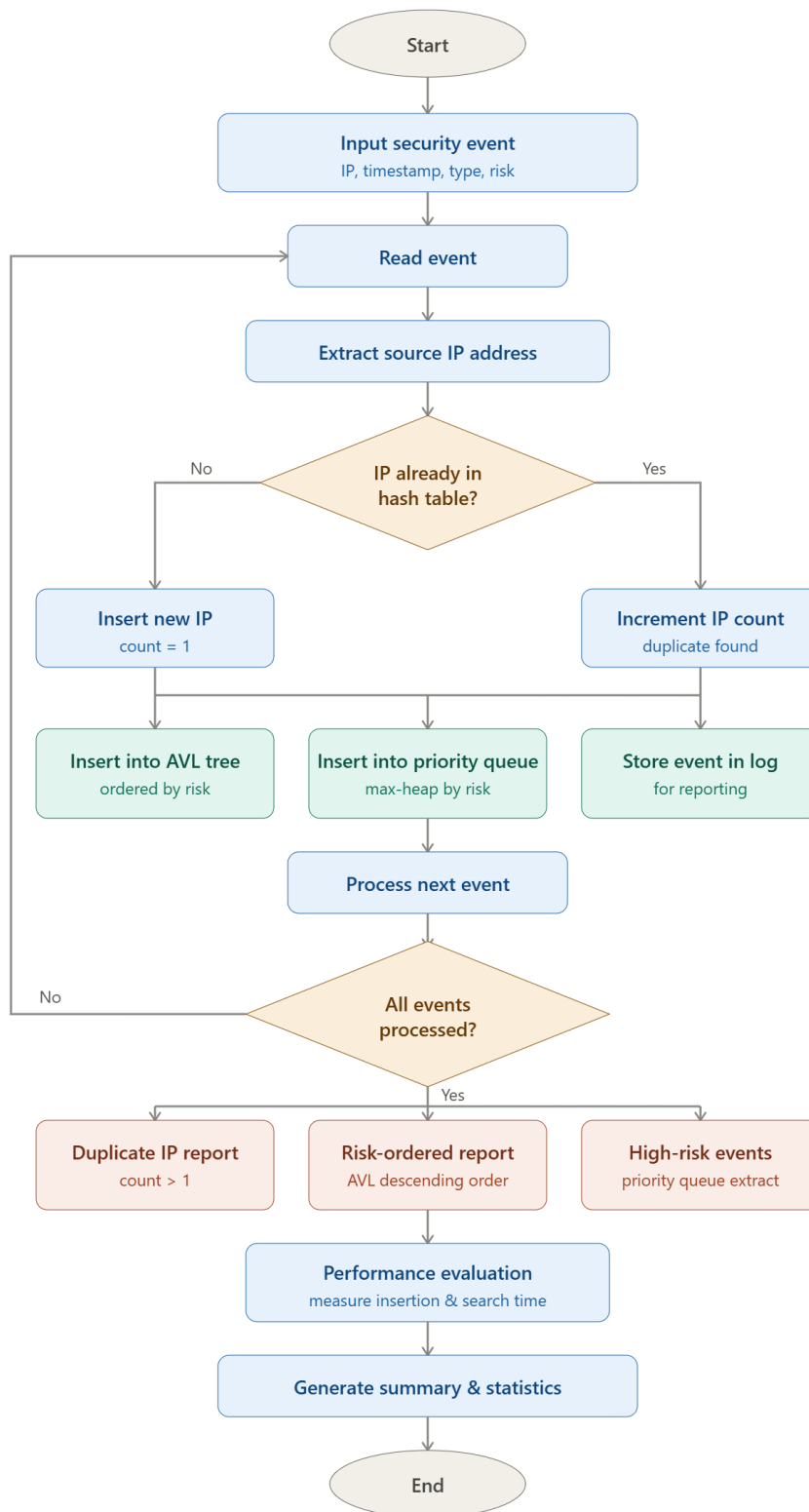
3. Solution / Design / Methodology

The system follows a sequential event-processing pipeline. Each event is inserted into the hash table, AVL tree and priority queue. The hash table records the number of occurrences of an IP. The AVL tree maintains events according to risk score. The max heap places the highest-risk event at the root.

Algorithm:

1. Initialize the hash table, AVL tree and priority queue.
2. Read each security event.
3. Calculate the hash index of the source IP.
4. Search the corresponding chain; increment the count if the IP already exists.
5. Otherwise create a new hash node.
6. Insert the event into the AVL tree using its risk score.
7. Insert the event into the max-heap priority queue.
8. Report IPs with more than one occurrence.
9. Traverse the AVL tree in descending risk order.

10. Repeatedly remove the maximum element from the priority queue.



Implementation Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define TABLE_SIZE 1009
#define MAX_EVENTS 10000

typedef struct {
    char ip[20], timestamp[30], type[30];
    int risk;
} Event;

typedef struct HashNode {
    char ip[20];
    int count;
    struct HashNode *next;
} HashNode;

HashNode *hashTable[TABLE_SIZE];

unsigned int hashFunction(char *ip) {
    unsigned int h = 0;
    while (*ip) h = h * 31 + *ip++;
    return h % TABLE_SIZE;
}

void insertHash(char *ip) {
    int i = hashFunction(ip);
    HashNode *p = hashTable[i];
    while (p) {
        if (strcmp(p->ip, ip) == 0) { p->count++; return; }
        p = p->next;
    }
    HashNode *n = malloc(sizeof(HashNode));
    strcpy(n->ip, ip);
    n->count = 1;
    n->next = hashTable[i];
    hashTable[i] = n;
}

int searchHash(char *ip) {
    HashNode *p = hashTable[hashFunction(ip)];
    while (p) {
        if (strcmp(p->ip, ip) == 0) return p->count;
        p = p->next;
    }
    return 0;
}

typedef struct AVLNode {
    Event event;
```

```

    int height;
    struct AVLNode *left, *right;
} AVLNode;

```

```

int height(AVLNode *n) { return n ? n->height : 0; }
int max(int a, int b) { return a > b ? a : b; }

```

```

AVLNode *newAVL(Event e) {
    AVLNode *n = malloc(sizeof(AVLNode));
    n->event = e; n->height = 1; n->left = n->right = NULL;
    return n;
}

```

```

AVLNode *rightRotate(AVLNode *y) {
    AVLNode *x = y->left, *t = x->right;
    x->right = y; y->left = t;
    y->height = 1 + max(height(y->left), height(y->right));
    x->height = 1 + max(height(x->left), height(x->right));
    return x;
}

```

```

AVLNode *leftRotate(AVLNode *x) {
    AVLNode *y = x->right, *t = y->left;
    y->left = x; x->right = t;
    x->height = 1 + max(height(x->left), height(x->right));
    y->height = 1 + max(height(y->left), height(y->right));
    return y;
}

```

```

int balance(AVLNode *n) { return n ? height(n->left) - height(n->right) : 0; }

```

```

AVLNode *insertAVL(AVLNode *root, Event e) {
    if (!root) return newAVL(e);
    if (e.risk < root->event.risk) root->left = insertAVL(root->left, e);
    else root->right = insertAVL(root->right, e);
    root->height = 1 + max(height(root->left), height(root->right));
    int b = balance(root);
    if (b > 1 && e.risk < root->left->event.risk) return rightRotate(root);
    if (b < -1 && e.risk >= root->right->event.risk) return leftRotate(root);
    if (b > 1 && e.risk >= root->left->event.risk) {
        root->left = leftRotate(root->left);
        return rightRotate(root);
    }
    if (b < -1 && e.risk < root->right->event.risk) {
        root->right = rightRotate(root->right);
        return leftRotate(root);
    }
    return root;
}

```

```

void displayAVL(AVLNode *root) {
    if (!root) return;
    displayAVL(root->right);
    printf("IP: %-15s Risk: %3d Type: %s\n", root->event.ip, root->event.risk, root->event.type);
}

```

```

    displayAVL(root->left);
}

Event heap[MAX_EVENTS];
int heapSize = 0;

void swap(Event *a, Event *b) { Event t = *a; *a = *b; *b = t; }

void insertPriority(Event e) {
    int i = heapSize;
    heap[heapSize++] = e;
    while (i > 0) {
        int p = (i - 1) / 2;
        if (heap[p].risk >= heap[i].risk) break;
        swap(&heap[p], &heap[i]);
        i = p;
    }
}

Event deleteHighestRisk() {
    Event result = heap[0];
    heap[0] = heap[--heapSize];
    int i = 0;
    while (1) {
        int l = 2*i+1, r = 2*i+2, largest = i;
        if (l < heapSize && heap[l].risk > heap[largest].risk) largest = l;
        if (r < heapSize && heap[r].risk > heap[largest].risk) largest = r;
        if (largest == i) break;
        swap(&heap[i], &heap[largest]);
        i = largest;
    }
    return result;
}

void performanceTest() {
    int sizes[] = {100, 500, 1000, 2000, 5000};
    char ip[20];
    printf("\nEvents\tTime(sec)\n");
    for (int j = 0; j < 5; j++) {
        clock_t start = clock();
        for (int i = 0; i < sizes[j]; i++) {
            sprintf(ip, "192.168.1.%d", i % 255);
            insertHash(ip);
        }
        clock_t end = clock();
        printf("%d\t%.6f\n", sizes[j], (double)(end-start)/CLOCKS_PER_SEC);
    }
}

int main() {
    Event events[] = {
        {"192.168.1.10", "10:00:01", "Login Failed", 90},
        {"192.168.1.20", "10:00:05", "Port Scan", 80},
        {"192.168.1.10", "10:00:10", "Brute Force", 95},
    }
}

```

```

    {"10.0.0.5", "10:00:15", "Normal Login", 20},
    {"172.16.0.8", "10:00:20", "Malware Alert", 100},
    {"192.168.1.20", "10:00:25", "Port Scan", 85}
};
int n = sizeof(events)/sizeof(events[0]);
AVLNode *root = NULL;

for (int i = 0; i < n; i++) {
    insertHash(events[i].ip);
    root = insertAVL(root, events[i]);
    insertPriority(events[i]);
}

printf("=== DUPLICATE IPs ===\n");
for (int i = 0; i < n; i++)
    if (searchHash(events[i].ip) > 1)
        printf("%s -> %d occurrences\n", events[i].ip, searchHash(events[i].ip));

printf("\n=== AVL RISK REPORT ===\n");
displayAVL(root);

printf("\n=== HIGH RISK EVENTS ===\n");
while (heapSize) {
    Event e = deleteHighestRisk();
    printf("Risk: %3d | IP: %-15s | %s\n", e.risk, e.ip, e.type);
}

printf("\n=== PERFORMANCE TEST ===\n");
performanceTest();
return 0;
}

```

Output Screen Shot

```
===== REAL-TIME CYBER THREAT DETECTION ENGINE =====
Total Events Processed : 6

----- DUPLICATE IP DETECTION REPORT -----
192.168.1.10 -> 2 occurrences
192.168.1.20 -> 2 occurrences

----- AVL TREE : EVENTS ORDERED BY RISK (Highest Risk First) -----
IP Address | Risk | Event Type | Timestamp
172.16.0.8 | 100 | Malware Alert | 10:00:20
192.168.1.10 | 95 | Brute Force Login | 10:00:10
192.168.1.10 | 90 | Login Failed | 10:00:01
192.168.1.20 | 85 | Port Scan | 10:00:25
192.168.1.20 | 80 | Port Scan | 10:00:05
10.0.0.5 | 20 | Normal Login | 10:00:15

----- PRIORITY QUEUE : HIGH RISK EVENTS (Highest Risk First) -----
Rank | IP Address | Risk | Event Type | Timestamp
1 | 172.16.0.8 | 100 | Malware Alert | 10:00:20
2 | 192.168.1.10 | 95 | Brute Force Login | 10:00:10
3 | 192.168.1.10 | 90 | Login Failed | 10:00:01
4 | 192.168.1.20 | 85 | Port Scan | 10:00:25
5 | 192.168.1.20 | 80 | Port Scan | 10:00:05
6 | 10.0.0.5 | 20 | Normal Login | 10:00:15

----- HASH TABLE PERFORMANCE TEST (Insertion Time) -----
Number of Events | Insertion Time (seconds)
100 0.000054
500 0.000228
1000 0.000444
2000 0.000912
5000 0.002280

COMPLEXITY SUMMARY
Hash Table (Search/Insert) : O(1) average case
AVL Tree (Insert/Search) : O(log n)
Priority Queue (Insert) : O(log n)
Priority Queue (DeleteMax) : O(log n)

Program finished successfully.
```

4. Use of Modern Tools

Tool	Purpose
C / GCC	Program development and compilation
Visual Studio Code	Source-code editing and execution
GitHub	Version control and submission
Terminal	Compilation and execution
clock()	Execution-time measurement

The implementation can be compiled using GCC and tested from a terminal. Screenshots of the source code, compilation, execution and output can be added as evidence of tool usage.

5. Results and Validation

Sample Input

IP Address	Event Type	Risk
192.168.1.10	Login Failed	90
192.168.1.20	Port Scan	80
192.168.1.10	Brute Force	95
10.0.0.5	Normal Login	20
172.16.0.8	Malware Alert	100
192.168.1.20	Port Scan	85

Expected Duplicate Detection:

- 192.168.1.10 → 2 occurrences
- 192.168.1.20 → 2 occurrences

Expected Risk Ranking:

Rank	IP Address	Risk	Event
1	172.16.0.8	100	Malware Alert
2	192.168.1.10	95	Brute Force
3	192.168.1.10	90	Login Failed
4	192.168.1.20	85	Port Scan
5	192.168.1.20	80	Port Scan
6	10.0.0.5	20	Normal Login

Performance Test:

Number of Events	Measured Time (sec)
100	Record from program
500	Record from program
1000	Record from program
2000	Record from program
5000	Record from program

The execution-time values should be copied from the actual program output. The test demonstrates how hash insertion performance changes as the number of events increases.

6. Analysis and Engineering Decision

Approach	Advantage	Limitation
Linear Search	Simple	$O(n)$ search
Hash Table + Chaining	$O(1)$ average lookup	Collision chains can grow
Normal BST	Ordered storage	Can become unbalanced

AVL Tree	Guaranteed logarithmic height	Rotations add implementation complexity
Priority Queue	Fast highest-risk retrieval	Not intended for general ordered traversal

Hashing was selected for duplicate-IP detection because average lookup is $O(1)$. AVL was selected instead of an ordinary BST because it maintains balance and provides $O(\log n)$ operations. A max heap was selected for continuously retrieving the highest-risk event. Combining the three structures provides better functionality than relying on a single data structure.

7. Broader Considerations

Sustainability

Efficient algorithms reduce unnecessary computation and support scalable processing of large security-log streams.

Society and Safety

Rapid threat identification can help organizations protect digital services, users and infrastructure from malicious activity.

Ethics and Privacy

Security logs may contain sensitive information. Access should be restricted to authorized personnel, and collected data should be used only for legitimate security purposes.

Professional Responsibility

The system must be tested carefully because false positives can create unnecessary alerts while false negatives can allow threats to remain undetected.

8. Conclusion

The Real-Time Cyber Threat Detection Engine demonstrates the practical use of hashing, AVL trees and priority queues in cybersecurity. Hashing provides fast duplicate-IP detection, the AVL tree maintains risk-ordered events, and the priority queue surfaces the highest-risk events. Performance testing provides evidence of how the hash table behaves as the workload increases. The system can be extended in future with real-time log files, larger-scale datasets, multithreading and improved hash-table strategies.

9. Student Reflection

This assignment helped me understand how data structures can be combined to solve a practical cybersecurity problem. I learned the importance of hashing for fast lookup, chaining for collision handling, AVL rotations for maintaining balance, and priority queues for risk ranking. I also learned that choosing the correct data structure depends on the operation that needs to be optimized.

With additional time and resources, I would extend the project to process live security logs, compare chaining with open addressing, test larger datasets, add graphical performance analysis, and improve memory management.

10. References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C., Introduction to Algorithms, MIT Press, 2022.

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D., *Compilers: Principles, Techniques, and Tools*, Pearson, 2006.
- Weiss, M. A., *Data Structures and Algorithm Analysis in C*, Pearson, 2014.
- Sedgewick, R., & Wayne, K., *Algorithms*, Addison-Wesley, 2011.
- Knuth, D. E., *The Art of Computer Programming, Volume 3: Sorting and Searching*, Addison-Wesley, 1998.
- Black, P. E., *Data Structures and Algorithms*, National Institute of Standards and Technology (NIST).
- Stallings, W., *Network Security Essentials: Applications and Standards*, Pearson, 2017.
- Kurose, J. F., & Ross, K. W., *Computer Networking: A Top-Down Approach*, Pearson, 2021.
- NIST, *Computer Security Incident Handling Guide (SP 800-61)*, National Institute of Standards and Technology, 2012.
- NIST, *Cybersecurity Framework (CSF) 2.0*, National Institute of Standards and Technology, 2024.

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO2	PO1, PO2	L3/L4	10
Application of knowledge	CO2, CO3	PO1, PO2	L3/L4	20
Solution / Design / Implementation	CO3, CO4	PO3, PO5	L4/L5/L6	20
Modern tool usage	CO4, CO5	PO5	L3/L4	10
Validation	CO5	PO2, PO4	L4/L5	15
Analysis & justification	CO3, CO5	PO2, PO3	L4/L5	15
Broader considerations	CO5	PO6, PO7	L4/L5	5
Documentation & reflection	CO5	PO10	L3/L4	5

H. Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis:

Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____